BombLab 实验报告

第 1 关 函数调用与参数传递

输入格式与判定条件

phase_01 在调用 valid_u32_fields 前将第二个参数设为 1，随后用 sscanf 读取一个整数。sscanf 返回值与 1 比较，说明本关口令由一个无符号整数构成。

```
40202d: mov  $0x1,%esi
40203a: call valid_u32_fields
40205e: call __isoc99_sscanf@plt
402063: cmp  $0x1,%eax
402068: mov  %r12,%rdi
40206b: call p1_compute
402070: cmp  %eax,-0x14(%rbp)
402073: sete %al
```

challenge 在入口处从 %rsi 保存到 %r12。程序将它传给 p1_compute，再把返回值与输入整数比较。因此，需要恢复的是 p1_compute 的计算关系。

关键计算与推理

p1_compute 依次调用 p1_step2、p1_step0 和 p1_step1。先跟踪调用前的参数寄存器，再分析各小函数的运算。

```
# p1_compute 中的关键参数准备和调用
401fe5: mov  %rdi,%rax
401fe8: mov  %rdi,%rdx
401feb: mov  %edi,%esi
401fed: shr  $0x20,%rax
401ff1: mov  %edi,%r8d
401ff4: shr  $0x10,%rdx
401ff8: mov  %eax,%edi
401ffd: call p1_step2
402002: mov  %rdx,%rsi
402005: mov  %r8d,%edi
402008: mov  %eax,%ecx
40200a: call p1_step0
40200f: mov  %ecx,%esi
402011: mov  %eax,%edi
402013: call p1_step1
```

设 challenge 为 C，定义：

```
L = u32(C)
H = u32(C >> 32)
M = u32(C >> 16)
```

p1_step2 收到 H 和 L。其关键指令为：

```
lea (%rdi,%rsi,2),%eax
xor $0x1e40,%eax
```

lea 在这里计算第一个参数加上第二个参数的两倍，再与常数异或，得到：

```
A = u32(H + 2L) XOR 0x1e40
```

p1_step0 收到 L 和 M：

```
xor $0x88bf,%esi
lea (%rsi,%rdi,4),%eax
```

它先对 M 异或，再加上 L 的四倍：

```
B = u32((M XOR 0x88bf) + 4L)
```

最后，p1_step1 的第一个参数为 B，第二个参数为 A：

```
lea 0xabc9(%rsi),%eax
xor %edi,%eax
```

所以本关需要输出：

```
answer = u32(A + 0xabc9) XOR B
```

实现时将该结果存入 values[0]，设置 count = 1，由已有的 write_answer 处理输出格式。

第 2 关 数组 循环与递推

输入格式与首项

phase_02 将输入字段数设为 6，sscanf 返回值也需要等于 6。输入的六个整数依次存放在栈上的数组位置。程序先检查首项，再循环检查其余五项。

首项计算的关键指令为：

```
4020ea: mov    %r13,%rax
4020ed: mov    -0x40(%rbp),%esi
4020f0: shr    $0x5,%rax
4020f4: add    %eax,%eax
4020f6: movzbl %al,%eax
4020f9: add    $0x1d,%eax
4020fc: cmp    %eax,%esi
```

challenge 保存在 %r13。这段代码先右移 5 位，再乘 2，随后取结果的低 8 位，最后加上十进制 29。因此：

```
v[0] = ((2 × (C >> 5)) AND 0xff) + 29
```

取低 8 位发生在乘 2 之后，这个截断顺序必须保留。例如 seed 42 的 challenge 为 0x47526757130f9f52，按上述顺序计算的首项为 273。

循环关系与推理

循环开始前，%edx 设为 1，作为数组下标；%ecx 设为 0x17，即右移位数 23。

```
402100: mov $0x17,%ecx
402105: mov $0x1,%edx
40210a: mov %r13,%rax
40210d: lea (%rsi,%rsi,4),%esi
402110: shr %cl,%rax
402113: sub %edx,%esi
402115: and $0x1f,%eax
402118: add $0x9,%eax
40211b: xor %esi,%eax
40211d: mov -0x40(%rbp,%rdx,4),%esi
402121: cmp %eax,%esi
402125: add $0x1,%rdx
402129: add $0x3,%ecx
40212c: cmp $0x6,%rdx
402130: jne 40210a
```

lea 计算上一项的五倍，sub 减去当前下标。另一方面，程序把 challenge 右移相应位数，取低 5 位，再加 9。两部分异或后得到当前项的期望值。比较通过后，当前输入成为下一轮的上一项。

对于 i = 1, 2, 3, 4, 5，计算关系为：

```
s[i] = 23 + 3(i − 1)
P[i] = u32(5v[i − 1] − i)
T[i] = ((C >> s[i]) AND 0x1f) + 9
v[i] = P[i] XOR T[i]
```

右移位数依次为 23、26、29、32、35。实现中使用循环产生后五项，每轮更新上一项，最后设置 count = 6。

第 3 关 switch 分支与跳转表

两个输入与分支选择

phase_03 要求两个整数。第一个整数的检查如下：

```
40227c: mov %r12d,%edi
402281: and $0x7,%edi
402284: cmp %edi,-0x18(%rbp)
```

因此第一个数为 k = u32(C) AND 7，范围为 0～7。程序还构造一个 32 位参数 x：

```
402289: mov %r12,%rax
40228c: mov %r12,%rsi
40228f: shr $0x3,%rax
402293: shr $0x20,%rsi
402297: xor %eax,%esi
402299: call p3_dispatch
```

由此得到：

```
x = u32(C >> 3) XOR u32(C >> 32)
```

跳转表的恢复

p3_dispatch 把 k 保存到 %eax，将 x 移到 %edi，然后根据 k 查表跳转：

```
4021d4: mov %edi,%eax
4021d6: mov %esi,%edi
4021d8: cmp $0x7,%eax
4021db: ja  402225
4021dd: notrack jmp *0x4057e0(,%rax,8)
```

表项大小为 8 字节。使用以下命令读取八个目标地址：

```
gdb -q -batch -ex 'x/8a 0x4057e0' ./bomb
```

读取出的地址依次为 0x4021f8、0x402200、0x402208、0x402210、0x402218、0x402220、0x4021e8、0x4021f0。再对照这些地址处的 jmp，恢复出选择关系。不能直接把函数在反汇编中的排列顺序当作分支编号。

分支计算关系

各 p3_case 返回一个 32 位整数。其中，lea 用于乘加运算，shr $1 用于无符号右移，xor 用于按位异或。以 p3_case_4 为例：

```
mov %edi,%eax
shr $1,%edi
xor $0xc237,%eax
add %edi,%eax
ret
```

它计算 u32((x XOR 0xc237) + (x >> 1))。结合跳转表，八个分支的完整关系如下：

第一个数 k	目标函数	第二个数 y
0	p3_case_4	u32((x XOR 0xc237) + (x >> 1))
1	p3_case_2	u32(5x + 0xe11e)
2	p3_case_5	u32(4x) XOR 0xfd1b
3	p3_case_0	u32(2x) XOR 0x4f4e
4	p3_case_3	u32(5x + 0x424e1)
5	p3_case_1	u32((x XOR 0x835f) + (x >> 1))
6	p3_case_7	u32(3x + 0x4d42)

第一个数 k	目标函数	第二个数 y
7	p3_case_6	u32(5x + 0x48f8a)

实现使用 switch (k) 选择计算式，把 k、y 分别存入 values[0]、values[1]，设置 count = 2。

第 4 关 二分查找与递归返回

参数与停止条件

phase_04 要求一个整数。它将 challenge 的低 7 位作为目标值，将 0、127 作为区间端点，再传入由 challenge 变换得到的 key。

```
402373: shr $0x20,%rax
402377: shr $0x7,%rcx
40237b: and $0x7f,%edi
40237e: mov $0x7f,%edx
402383: xor %eax,%ecx
402385: call p4_walk
40238a: cmp %eax,-0x14(%rbp)
```

对应参数为：

```
target = u32(C) AND 0x7f
left = 0
right = 127
key = u32(C >> 7) XOR u32(C >> 32)
```

p4_walk 先把 target 保存到 %eax，再使用 %edi 计算区间中点。关键指令如下：

```
4022b4: mov %edi,%eax
4022b6: mov %edx,%edi
4022b8: sub %esi,%edi
4022ba: shr $1,%edi
4022bc: add %esi,%edi
4022be: cmp %eax,%edi
4022c0: je 402310
```

中点 m = left + (right - left) / 2，除法向下取整。目标等于中点时停止递归，执行：

```
402310: lea 0x93(%rdi),%eax
402316: xor %ecx,%eax
402318: ret
```

此时返回 u32(m + 147) XOR key。

左右分支与返回关系

目标小于中点时使用无符号跳转 jb 进入左分支，把右端点改为 m - 1；目标大于中点时，把左端点改为 m + 1。两条分支均再次调用 p4_walk。

```
# 保存本层的 m + key
4022c7: lea (%rdi,%rcx,1),%ebx

# 右分支
4022d2: lea 0x1(%rdi),%esi
4022d7: call p4_walk
4022dc: lea (%rax,%rax,4),%eax
4022df: xor %ebx,%eax
4022e6: xor $0x7,%eax

# 左分支
4022f0: lea -0x1(%rdi),%edx
4022f5: call p4_walk
4022fa: lea (%rax,%rax,4),%eax
4022fd: xor %ebx,%eax
402304: xor $0x9,%eax
```

设下一层返回值为 child，当前层返回值为：

```
左分支：u32(5 × child) XOR u32(m + key) XOR 9
右分支：u32(5 × child) XOR u32(m + key) XOR 7
```

查找阶段不断缩小区间，返回阶段则按相反顺序组合每一层的结果。

实现方式

实际解法在 solve_phase_4 内用循环模拟递归。查找过程中，用 mids 数组保存每层中点，用 branch_xor 保存左分支的 9 或右分支的 7。找到目标后先计算基础返回值，再倒序处理记录，逐层执行乘 5 和异或。

初始区间为 0～127，最多记录 7 个非终止层，长度为 8 的数组足以保存路径。最终结果存入 values[0]，设置 count = 1。

第 5 关 结构体布局与链表遍历

两个结果的来源

phase_05 要求两个整数。它向 p5_traverse 传入 challenge 和一个局部变量的地址：

```
40244c: lea -0x14(%rbp),%rsi
402450: mov %r12,%rdi
402453: call p5_traverse
402458: mov %eax,%edx
40245c: cmp %edx,-0x1c(%rbp)
402461: mov -0x14(%rbp),%eax
402464: cmp %eax,-0x18(%rbp)
```

第一个输入与函数返回值比较，第二个输入与经地址参数写回的值比较。这说明遍历函数同时产生累计值和另一个结果。

节点字段与固定数据

遍历代码中的三种内存访问确定了节点字段的位置：

节点内偏移	字段	大小
0	value	4 字节
4	tag	4 字节
8	next 指针	8 字节

```
4023c7: mov 0x4(%rdx),%esi
4023cc: xor (%rdx),%eax
4023e4: mov 0x8(%rdx),%rdx
```

%rdx 是当前节点地址，mov 0x8(%rdx),%rdx 把 next 指向的节点设为当前节点。使用 GDB 读取起点表和节点区：

```
gdb -q -batch -ex 'x/8a 0x405b20' ./bomb
gdb -q -batch -ex 'x/48wx 0x405b60' ./bomb
```

节点区起始地址为 0x405b60，节点间隔为 16 字节。将地址转换为编号后，固定数据如下：

节点编号	value	tag	next 编号
0	0xb843	0x0450	10
1	0x289f	0x057d	9
2	0x0dc0	0x0a79	5
3	0x3215	0x073c	2
4	0x31c6	0x0026	8
5	0x757a	0x0233	11
6	0xadc8	0x035e	4
7	0xb614	0x064f	6
8	0x9500	0x0934	1
9	0x9a40	0x0b31	0
10	0x8dc6	0x0870	无
11	0x726c	0x0135	7

八个起始节点按表下标 0～7 排列，编号依次为：

```
heads = [3, 2, 5, 11, 7, 6, 4, 8]
```

遍历次数与累计关系

起点由 challenge 的低 3 位选择。(C >> 3) AND 3 决定处理起始节点后额外前进的次数，所以总共访问 1～4 个节点。

```
4023ad: and $0x7,%eax
4023b0: shr $0x3,%rcx
4023b4: mov 0x405b20(,%rax,8),%rdx
4023bf: shr $0x20,%rdi
4023c3: shr $0x6,%rax
4023ca: xor %edi,%eax
4023cc: xor (%rdx),%eax
4023d0: xor $0x1998,%eax
4023d5: mov %esi,(%r8)
4023d8: add %esi,%eax
4023da: and $0x3,%edi
```

将首个节点和后续节点的相同处理合并，可以写成：

```
current = heads[C AND 7]
visit_count = 1 + ((C >> 3) AND 3)
acc = u32(C >> 6) XOR u32(C >> 32) XOR 0x1998

每访问一个节点：
  acc = u32((acc XOR value[current]) + tag[current])
  last_tag = tag[current]
  按 next 进入下一节点
```

mov %esi,(%r8) 每轮覆盖写回当前 tag，所以第二个结果是最后访问节点的 tag。实现将固定字段和 next 编号记录为数组，根据 challenge 动态选择并遍历，输出 acc、last_tag，设置 count = 2。

第 6 关 函数指针表与间接调用

两次调用的组织方式

phase_06 要求一个整数，并将其与 p6_dispatch 的返回值比较。后者从同一张函数表选择两个函数顺序执行：

```
4024fc: shr $0x6,%rdi
402503: shr $0x20,%rax
402507: xor %eax,%edi
402510: and $0x38,%ebx
402513: and $0x7,%eax
402516: call *0x405dc0(,%rax,8)
40251d: xor $0xd7ff,%eax
402522: mov %eax,%edi
402524: call *0x405dc0(%rbx)
40252f: add $0x80e6,%eax
```

初始输入 x = u32(C >> 6) XOR u32(C >> 32)。第一次表下标是 C AND 7，第二次表下标是 (C >> 3) AND 7。第二次调用使用的 C AND 0x38 已经是 8 字节表项的字节偏移，因此汇编中不再额外乘 8。

第一次调用结果先与 0xd7ff 异或，再交给第二个函数；第二次返回后加上 0x80e6。

函数表与具体运算

用以下命令读取函数表：

```
gdb -q -batch -ex 'x/8a 0x405dc0' ./bomb
```

表下标	目标函数
0	p6_op_1
1	p6_op_2
2	p6_op_4
3	p6_op_0
4	p6_op_5
5	p6_op_4
6	p6_op_3
7	p6_op_4

表下标 2、5、7 指向同一个函数。逐个分析目标函数后，得到以下关系，其中乘加结果均按 32 位保留：

函数	返回值
p6_op_0(x)	u32(4x + 0xa697) XOR u32(7x)
p6_op_1(x)	u32((x XOR 0x1aa4) + 4x)
p6_op_2(x)	u32(((x >> 2) XOR x) + 0x1ec4)
p6_op_3(x)	ROL32(x, 1) XOR 0x93eb
p6_op_4(x)	u32(x + 0x6bd7) XOR (x >> 2)
p6_op_5(x)	ROR32(x, 1)

以 op0 为例，lea 先计算 8x，sub 减去 x 得到 7x，再与另一部分 4x + 0xa697 异或。op3 的关键指令为：

```
4024c4: mov %edi,%eax
4024c6: rol $1,%eax
4024c8: xor $0x93eb,%eax
```

循环移位需要保留移出的一位。对 uint32_t x，ROL32(x, 1) 对应 (x << 1) | (x >> 31)，ROR32(x, 1) 对应 (x >> 1) | (x << 31)。op3 的返回值还必须包含 XOR 0x93eb。

实现方式

在 solve_phase_6 内保存函数映射 [1, 2, 4, 0, 5, 4, 3, 4]，用两轮循环和 switch 实现对应运算。第一轮结束后异或 0xd7ff，第二轮结束后加 0x80e6。最终结果写入 values[0]，设置 count = 1。

Honor 输入布局与 challenge context 的组合校验

输入布局

Honor 在同一次运行的六关通过后读取第七行。solve_secret 接收 challenge 和 context；context 由前六行口令生成，由驱动计算后传入。

secret_gate 先检查长度：

```
4026dc: call strlen@plt
4026e1: sub $0x24,%rax
4026e5: cmp $0x7f,%rax
4026e9: jbe 402700
```

无符号比较要求字符串长度在 36～163 字节之间，不包含末尾的空字符。secret_copy 按字节原样复制输入，直到复制到空字符结束：

```
402690: movzbl (%rsi,%rax,1),%edx
402694: mov %dl,(%rdi,%rax,1)
402697: add $0x1,%rax
40269b: test %dl,%dl
40269d: jne 402690
```

复制起点为 rbp - 0xd0。随后检查 rbp - 0xb0 的四个字节，并将 rbp - 0xac 作为后续校验字符串的起点：

```
40270b: cmpl $0x4e45504f,-0xb0(%rbp)
402715: jne 4026eb
40271d: lea -0xac(%rbp),%rdi
402724: call secret_phase
```

两个位置距复制起点分别为 32、36 字节。小端序下，0x4e45504f 的内存字节为 4f 50 45 4e，即 OPEN。因此第七行布局为：

从 0 开始的偏移	内容
0～31	32 字节填充
32～35	OPEN
36 开始	三个空格分隔的无符号十进制整数

实现采用 32 个字符 0 填充，然后输出 OPEN 和三个整数。数据在局部对象中的相邻位置形成上述布局，解法按偏移构造字符串。

三个整数的计算

secret_phase 要求三个整数。设 Honor 的 challenge 为 C，context 为 T。第一个整数来自 context 的低两位：

```
402620: mov %r13d,%ecx
402625: and $0x3,%ecx
402628: cmp %ecx,-0x2c(%rbp)
```

定义以下 32 位中间量：

```
k = u32(T) AND 3
A = u32(C >> 32) XOR u32(C)
B = u32(T >> 32) XOR u32(T)
R = u32(k × 0xdae1) XOR u32(T >> 16)
```

secret_mix 用乘加和异或构造混合结果：

```
4025a4: lea 0x0(,%rdi,8),%eax
4025ab: sub %edi,%eax
4025ad: add $0xdae1,%eax
4025b2: xor $0x3aff,%eax
```

即 mix(x) = u32(7x + 0xdae1) XOR 0x3aff。该函数只改动 %eax，调用者后续继续使用的 %esi、%ecx、%r8d 和 %r9d 保持原值。三个输入依次为：

```
first = k
second = mix(A XOR B) XOR R
third = second XOR u32(B + k + 0x3aff + (A >> 5))
```

第三个数的关键计算是：

```
402661: shr $0x5,%esi
402664: lea 0x3aff(%r8,%rcx,1),%edx
40266c: add %esi,%edx
40266e: xor %edx,%eax
402670: cmp -0x24(%rbp),%eax
```

输出与验证方式

使用 snprintf 把整行写入 out，格式为：

```
"%032uOPEN%" PRIu32 " %" PRIu32 " %" PRIu32
```

第一个格式参数为 0u，所以 %032u 恰好生成 32 个填充字符。随后依次输出 k、second、third。检查 snprintf 返回值，格式化成功且未截断时返回 0。字符串自身不添加换行，由驱动处理行结束。

验证时，先生成前六行，再生成第七行，一起传给同一个 seed 的炸弹：

```
{ ./defuser --seed 43; ./defuser --secret --seed 43; } | NO_COLOR=1 ./bomb --seed 43
```

编译与验证记录

公开测试

各关实现均在保存后重新编译。使用完整公开测试检查六关和 Honor：

```
make test 2>&1 | tee evidence/public_all.txt | tail -n 15
```

每个正式关卡的 10 个公开 seed 均为 PASS，Honor 的 10 个公开 seed 也均为 PASS。

项目	通过数	总数
第 1 关	10	10
第 2 关	10	10
第 3 关	10	10
第 4 关	10	10
第 5 关	10	10
第 6 关	10	10
正式关卡合计	60	60
Honor	10	10

公开列表以外的 seed 验证

tests/public.json 的公开 seed 为：

```
0, 1, 7, 42, 99, 123, 2026, 65535, 987654321,
18446744073709551615
```

选择不在列表中的 seed 43，运行六关及 Honor，完整命令为：

```
{ ./defuser --seed 43; ./defuser --secret --seed 43; } | NO_COLOR=1 ./bomb --seed 43 | tee evidence/all_seed43.txt
```

该次运行的各关 challenge 和结果如下：

关卡	challenge	结果
第 1 关	0x9cde98852e60034b	Phase 1 defused.
第 2 关	0x6ec624aa85685867	Phase 2 defused.
第 3 关	0xd49ffe504bfdab7f	Phase 3 defused.
第 4 关	0x006ff81b72d591ff	Phase 4 defused.
第 5 关	0x18e47d0995aecbfe	Phase 5 defused.
第 6 关	0x523fb4dd5d162e88	Phase 6 defused.
Honor	程序未直接打印	Secret phase defused.

Honor 在同一次六关通过后完成校验，同时验证了前六行口令对应的 context。

运行结果截图

图 1 seed 43 下完整七关运行结果及公开测试汇总

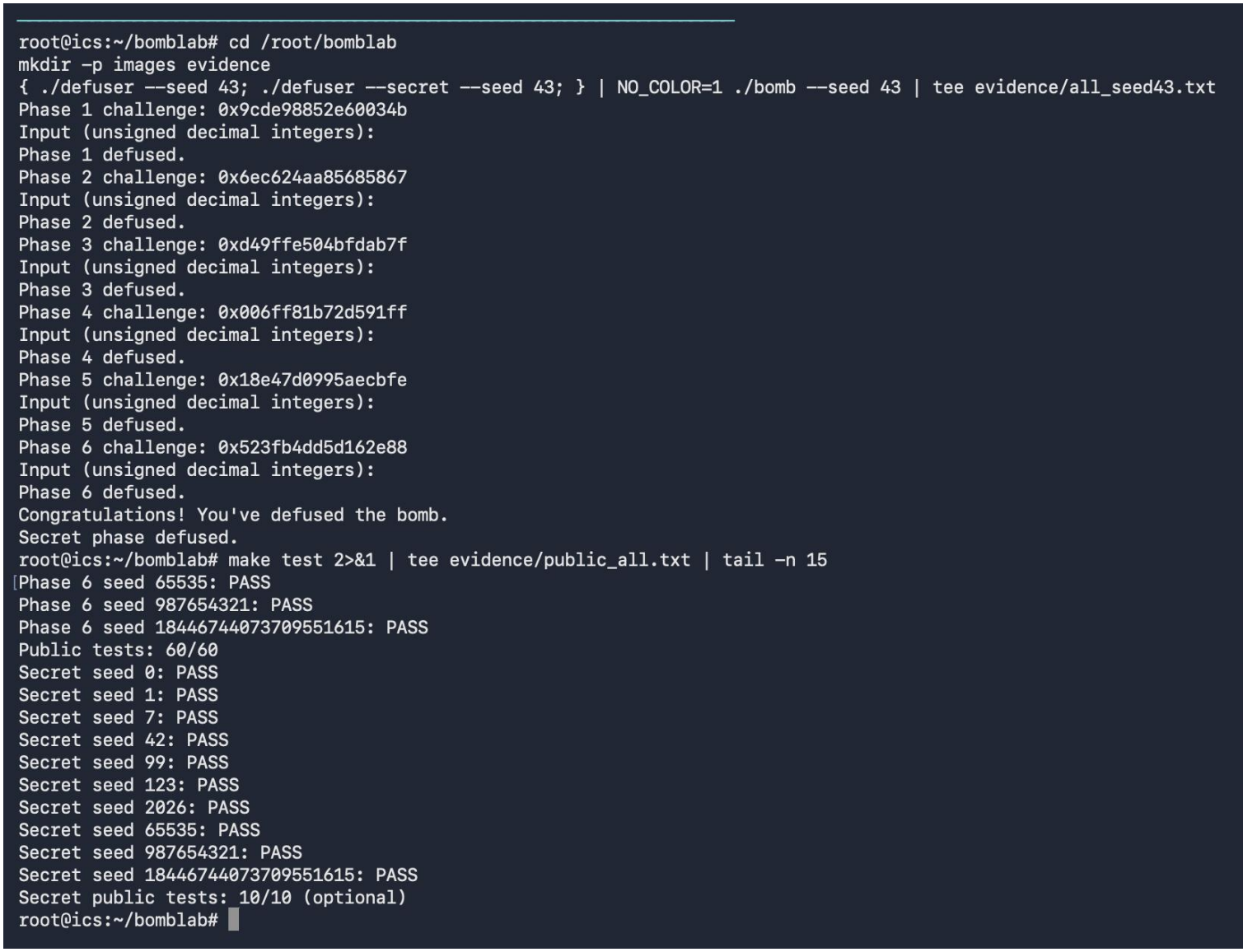


图 2a 正式第 1 至第 3 关公开测试逐项结果

```
python3 public_test.py
Phase 1 seed 0: PASS
Phase 1 seed 1: PASS
Phase 1 seed 7: PASS
Phase 1 seed 42: PASS
Phase 1 seed 99: PASS
Phase 1 seed 123: PASS
Phase 1 seed 2026: PASS
[Phase 1 seed 65535: PASS
Phase 1 seed 987654321: PASS
Phase 1 seed 18446744073709551615: PASS
Phase 2 seed 0: PASS
Phase 2 seed 1: PASS
Phase 2 seed 7: PASS
Phase 2 seed 42: PASS
Phase 2 seed 99: PASS
Phase 2 seed 123: PASS
Phase 2 seed 2026: PASS
Phase 2 seed 65535: PASS
Phase 2 seed 987654321: PASS
[Phase 2 seed 18446744073709551615: PASS
Phase 3 seed 0: PASS
Phase 3 seed 1: PASS
Phase 3 seed 7: PASS
Phase 3 seed 42: PASS
Phase 3 seed 99: PASS
Phase 3 seed 123: PASS
Phase 3 seed 2026: PASS
Phase 3 seed 65535: PASS
Phase 3 seed 987654321: PASS
Phase 3 seed 18446744073709551615: PASS
```

图 2b 正式第 4 至第 6 关公开测试逐项结果及汇总

```
Phase 4 seed 0: PASS
Phase 4 seed 1: PASS
Phase 4 seed 7: PASS
Phase 4 seed 42: PASS
Phase 4 seed 99: PASS
Phase 4 seed 123: PASS
Phase 4 seed 2026: PASS
Phase 4 seed 65535: PASS
Phase 4 seed 987654321: PASS
Phase 4 seed 18446744073709551615: PASS
Phase 5 seed 0: PASS
Phase 5 seed 1: PASS
Phase 5 seed 7: PASS
Phase 5 seed 42: PASS
Phase 5 seed 99: PASS
Phase 5 seed 123: PASS
Phase 5 seed 2026: PASS
Phase 5 seed 65535: PASS
Phase 5 seed 987654321: PASS
Phase 5 seed 18446744073709551615: PASS
Phase 6 seed 0: PASS
Phase 6 seed 1: PASS
Phase 6 seed 7: PASS
Phase 6 seed 42: PASS
Phase 6 seed 99: PASS
Phase 6 seed 123: PASS
Phase 6 seed 2026: PASS
Phase 6 seed 65535: PASS
Phase 6 seed 987654321: PASS
Phase 6 seed 18446744073709551615: PASS
Public tests: 60/60
```

参考资料与辅助工具

无特定参看资料，与 ai 不断对话逐步完成

使用 Codex 辅助理解汇编指令和函数调用关系、推导计算逻辑、以及测试记录。